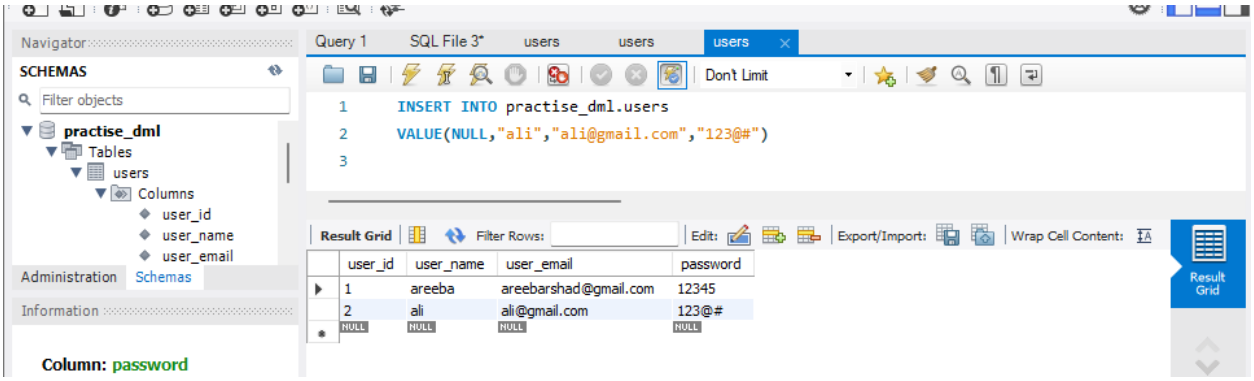
MySQL DML

1. Data Manipulation Language (DML)

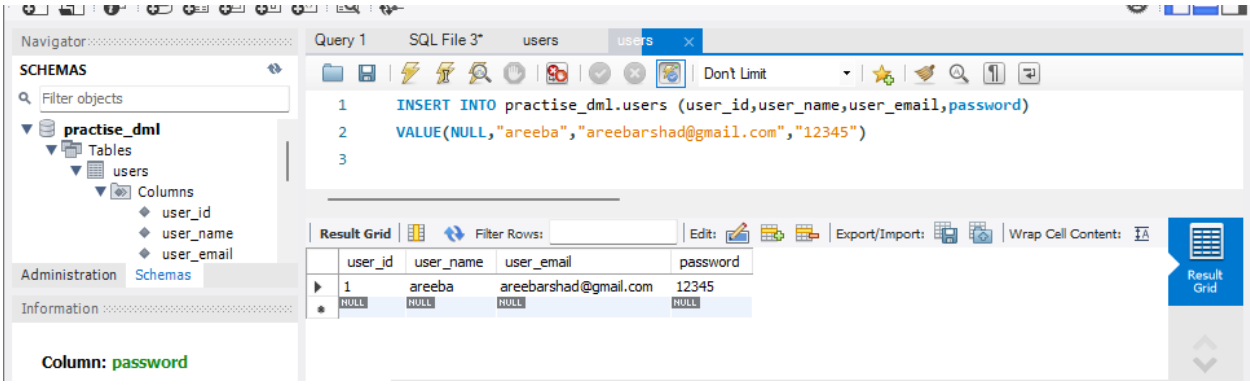
1.1 INSERT

- Used to insert rows into a table.
- **Syntax without specifying columns:**



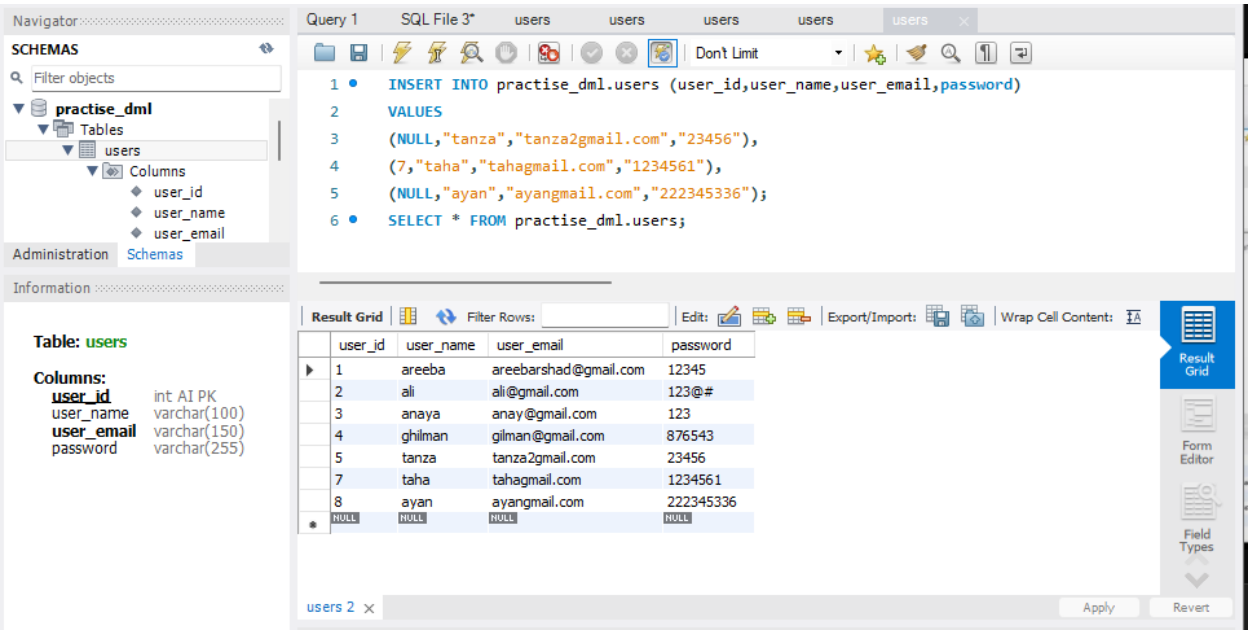
⚠️ Ensure data is provided in the **exact order** of table columns.

- **Syntax with specifying columns:**



You can insert values in any column order as long as you match the column names with the corresponding values.

For Multiple Insertions:



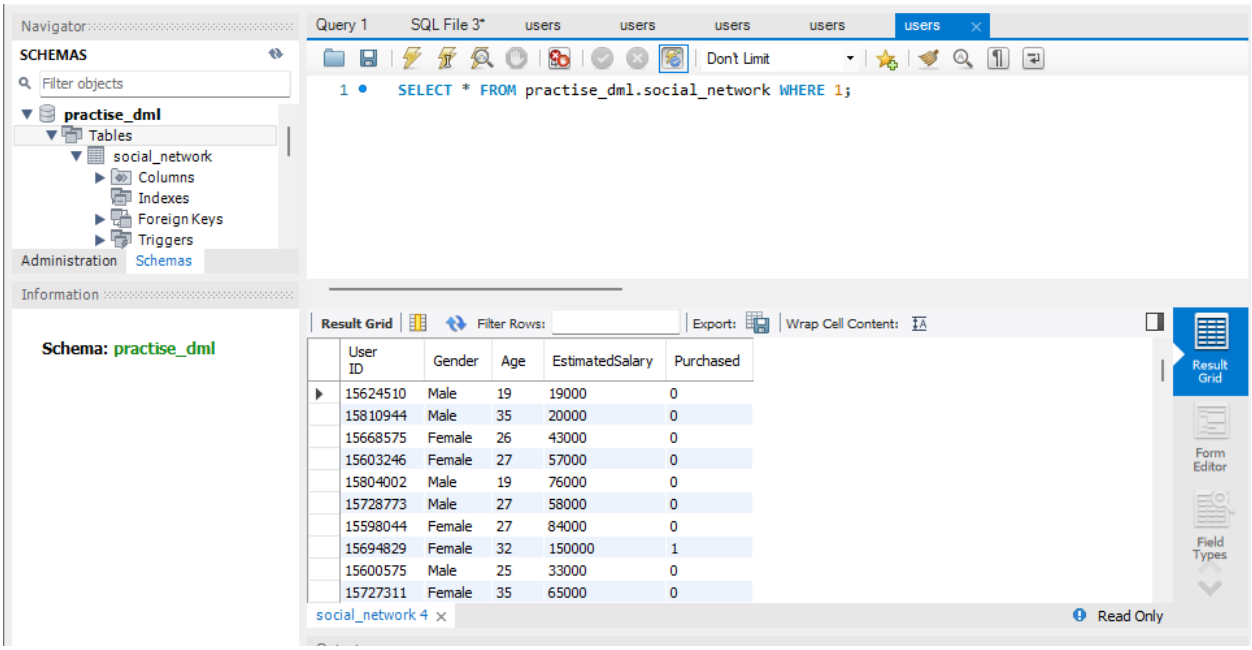
1.2 Import Data into MySQL Workbench

Steps:

1. Open table in Workbench.
2. Choose **Import Data** from options.
3. Map CSV/Excel columns to table columns.
4. Execute import.

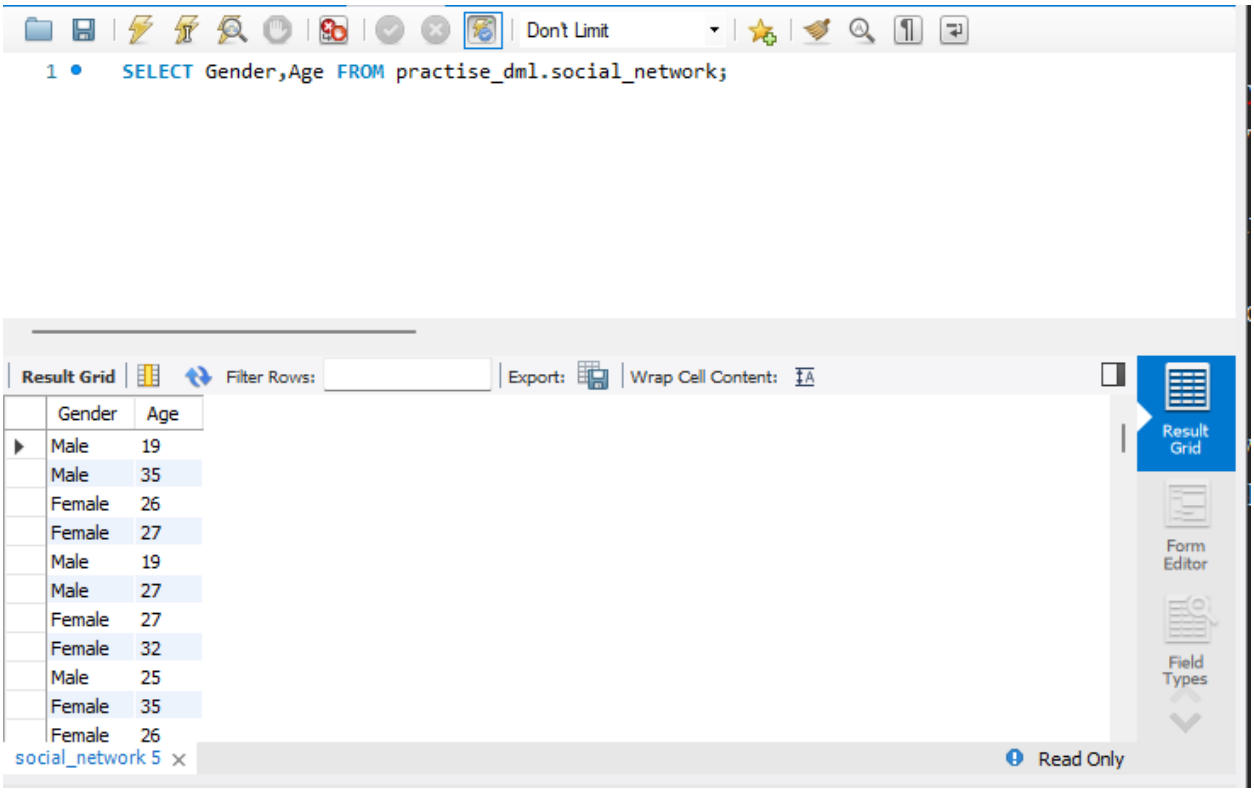
1.3 SELECT (Retrieve Data)

a) Retrieve All Rows

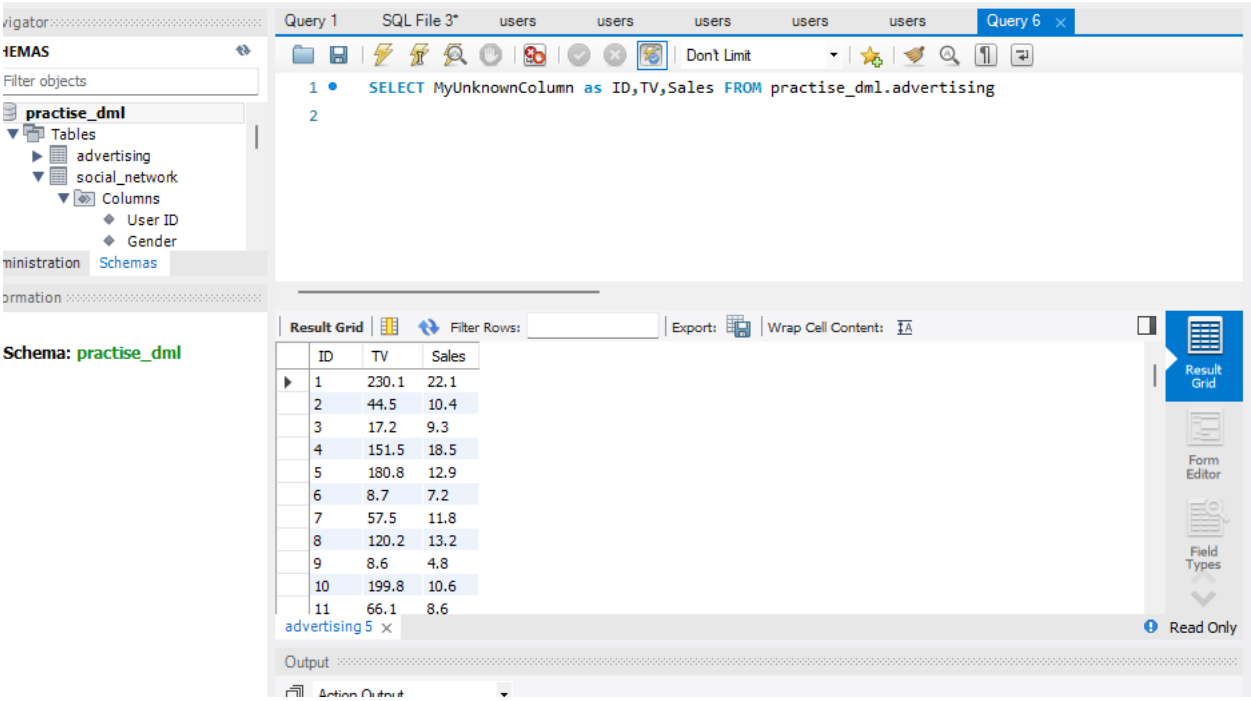


- `WHERE 1` is optional but indicates **no condition is applied**.

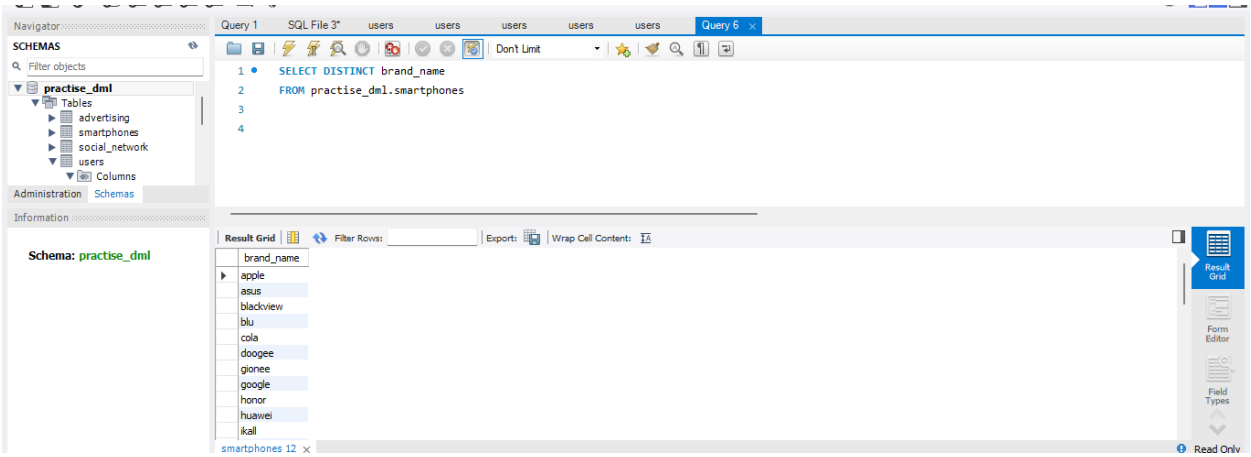
b) Select Specific Columns



- Use **aliases** to rename columns:



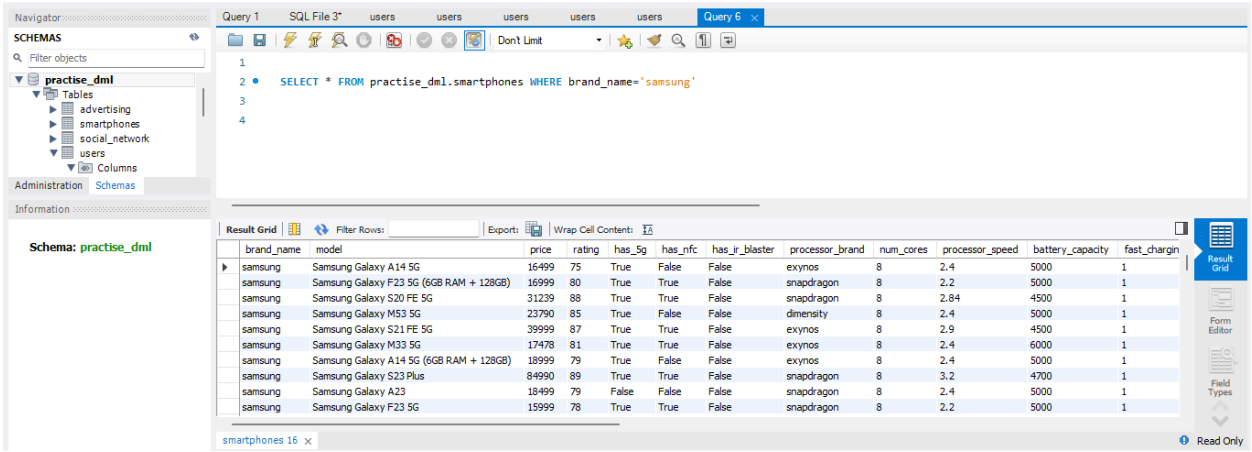
c) Remove Duplicate Values



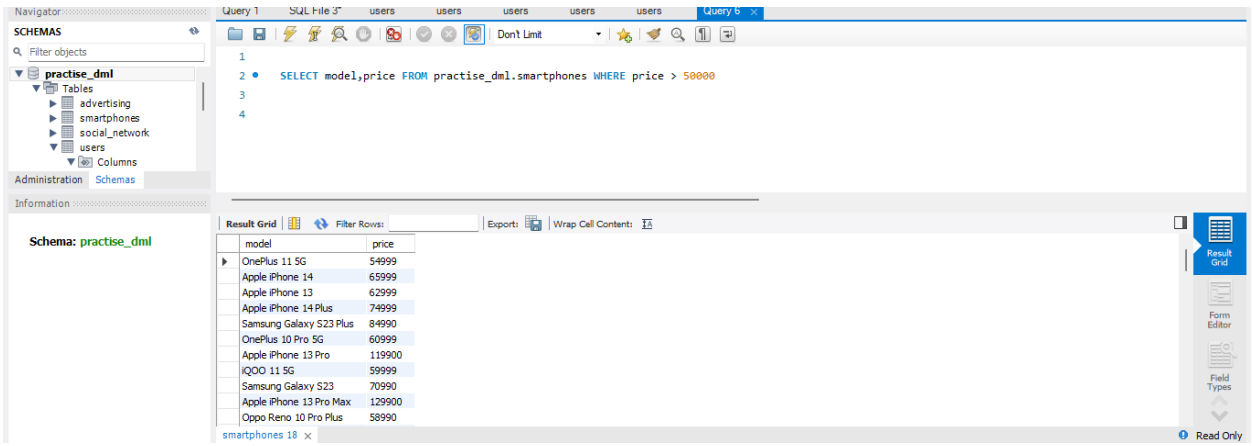
1.4 Filter Rows

a) By Condition

- All Samsung brand phones:

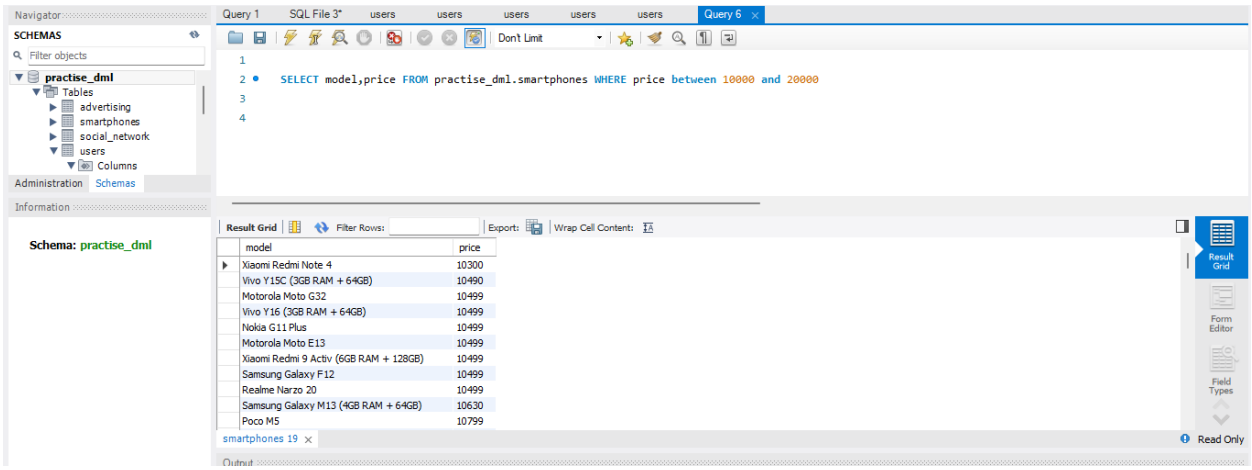


- Phones costing more than 50,000:



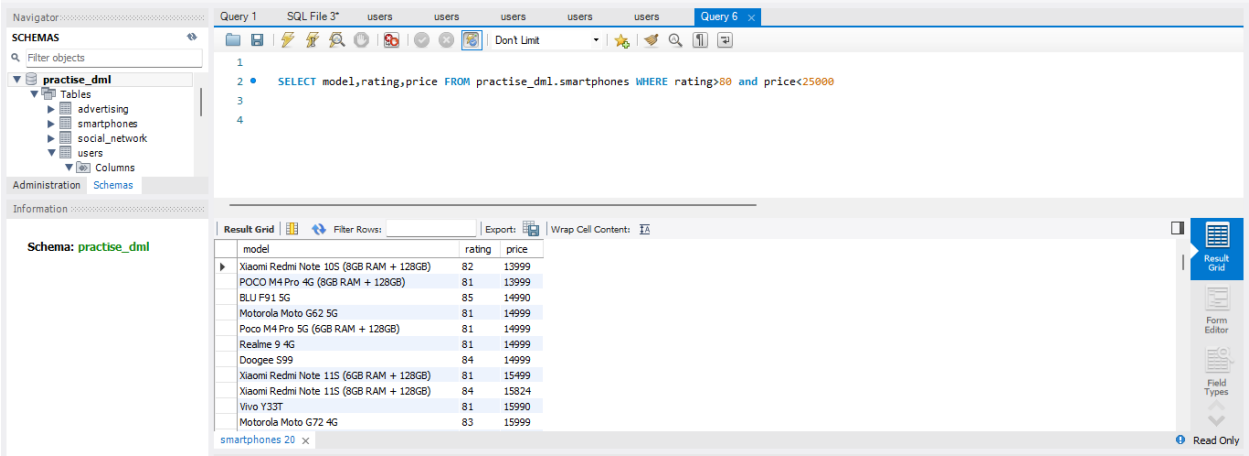
b) Using BETWEEN

- Phones priced between 10,000 and 20,000:

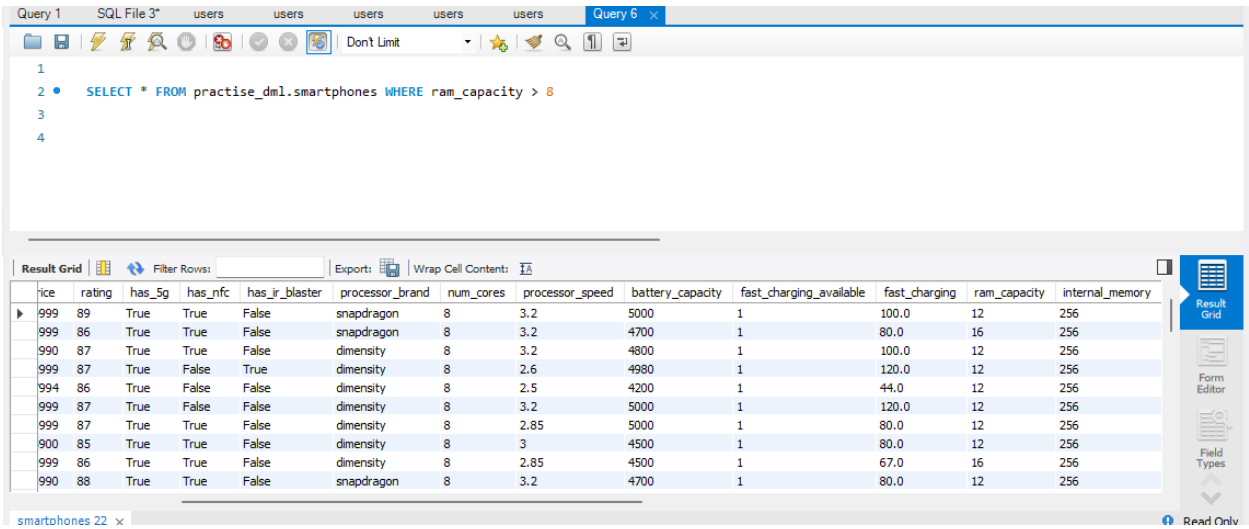


c) Using Multiple Conditions

- Phones with rating > 80 and price < 25,000:



- Samsung phones with RAM > 8GB:



- Samsung phones with Snapdragon processor:

```sql
SELECT brand_name,model,processor_brand FROM practise_dml.smartphones
WHERE brand_name='samsung' and processor_brand='snapdragon'
```

brand_name	model	processor_brand
samsung	Samsung Galaxy F23 5G (6GB RAM + 128GB)	snapdragon
samsung	Samsung Galaxy S20 FE 5G	snapdragon
samsung	Samsung Galaxy S23 Plus	snapdragon
samsung	Samsung Galaxy A23	snapdragon
samsung	Samsung Galaxy F23 5G	snapdragon
samsung	Samsung Galaxy A23 5G	snapdragon
samsung	Samsung Galaxy S22 5G	snapdragon
samsung	Samsung Galaxy S23	snapdragon
samsung	Samsung Galaxy F24 5G	snapdragon
samsung	Samsung Galaxy M54 5G	snapdragon
samsung	Samsung Galaxy S22 Plus 5G	snapdragon

d) Using IN / NOT IN

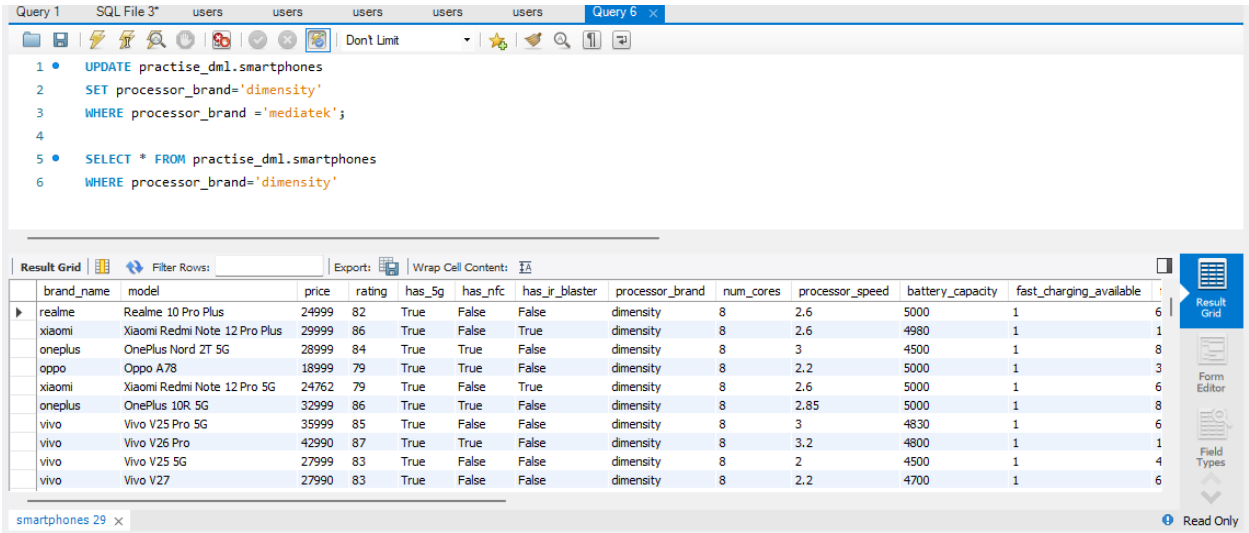
- Brands with specific processors:

```sql
SELECT brand_name,processor_brand FROM practise_dml.smartphones
WHERE processor_brand IN ('snapgarden','exynos','bionic')
```

brand_name	processor_brand
samsung	exynos
apple	bionic
apple	bionic
apple	bionic
samsung	exynos
samsung	exynos
samsung	exynos
apple	bionic
apple	bionic
samsung	exynos
samsung	exynos

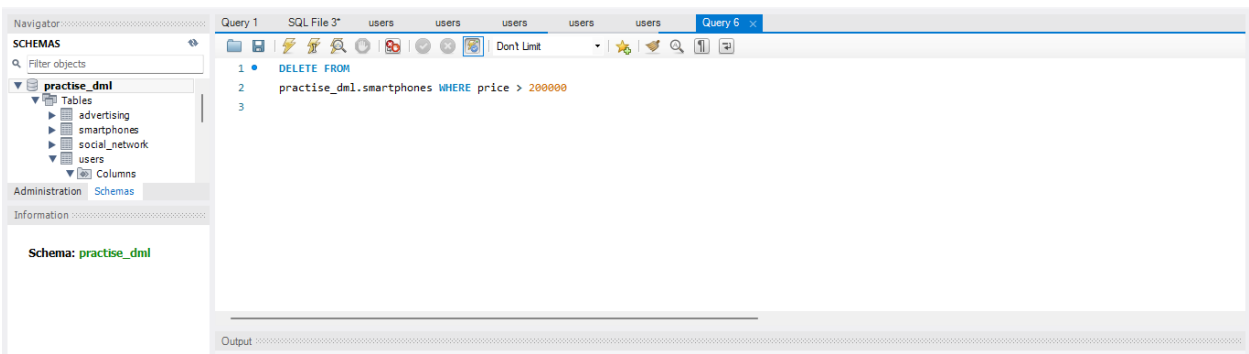
1.5 UPDATE

- Used to update specific fields in rows.



1.6 DELETE

- Used to delete rows.



2. Functions in SQL

- A **function** is a reusable piece of code to perform calculations or operations.

2.1 Types of Functions

1. **Built-in functions**
   - **Scalar**: Operates on each value individually.
   - **Aggregate**: Returns a single summary value for multiple rows.
2. **User-defined functions**: Custom functions created by users.

2.2 Aggregate Functions

Function	Description	Example
`MAX()`	Returns maximum value	`SELECT MAX(price) FROM phones;`

`MIN()`	Returns minimum value	`SELECT MIN(price) FROM phones;`
`AVG()`	Returns average	`SELECT AVG(rating) FROM phones WHERE brand='Apple';`
`SUM()`	Returns sum	`SELECT SUM(price) FROM phones;`
`COUNT()`	Returns count	`SELECT COUNT(*) FROM phones WHERE brand='OnePlus';`
`COUNT(DISTINCT)`	Count unique values	`SELECT COUNT(DISTINCT brand) FROM phones;`
`STD()`	Returns standard deviation	`SELECT STD(screen_size) FROM phones;`
`VAR()`	Returns variance	`SELECT VAR(price) FROM phones WHERE brand='Xiaomi';`

2.3 Scalar Functions

Function	Description	Example
`ABS()`	Returns absolute value	`SELECT ABS(rating-AVG(rating)) FROM phones WHERE brand='Samsung';`
`ROUND()`	Rounds a number to n decimals	`SELECT ROUND(ppi, 2) FROM phones;`
`CEIL()`	Returns smallest integer ≥ number	`SELECT CEIL(screen_size) FROM phones;`
`FLOOR()`	Returns largest integer ≤ number	`SELECT FLOOR(screen_size) FROM phones;`

3. Notes on Query Execution Order

1. **FROM (F)** – Specifies table
2. **JOIN (J)** – Combines tables
3. **WHERE (W)** – Filters rows
4. **GROUP BY (G)** – Groups rows
5. **HAVING (H)** – Filters groups
6. **SELECT (S)** – Chooses columns
7. **DISTINCT (D)** – Removes duplicates
8. **ORDER BY (O)** – Sorts results

Reference: infitq.com